Time Series Decomposition and Exponential Smoothing

Gianmaria Monaco — Bocconi University — Time Series Analysis — Group 32

1 Introduction

This report explores trend, seasonality, and exponential smoothing through simulated data and four time series available in R: AirPassengers, co2, UKgas, and Nile. An additional application examines Italian COVID-19 data.

The analysis compares decomposition on original and logarithmic scales and illustrates how the smoothing parameter affects simple exponential smoothing.

2 Creating Time Series Objects

We simulate 24 independent observations from a normal distribution with mean zero and variance two, then round them to two decimal places. We use the same observations to create two time series objects with different frequencies.

```
x <- round(rnorm(24, mean = 0, sd = sqrt(2)), 2)

y <- ts(x)
z <- ts(x, frequency = 12)

y
```

```
## Time Series:
## Start = 1
## End = 24
## Frequency = 1
##  [1]  1.94 -0.80  0.51  0.90  0.57 -0.15  2.14 -0.13  2.85 -0.09  1.85  3.23
## [13] -1.96 -0.39 -0.19  0.90 -0.40 -3.76 -3.45  1.87 -0.43 -2.52 -0.24  1.72
```

```
z
```

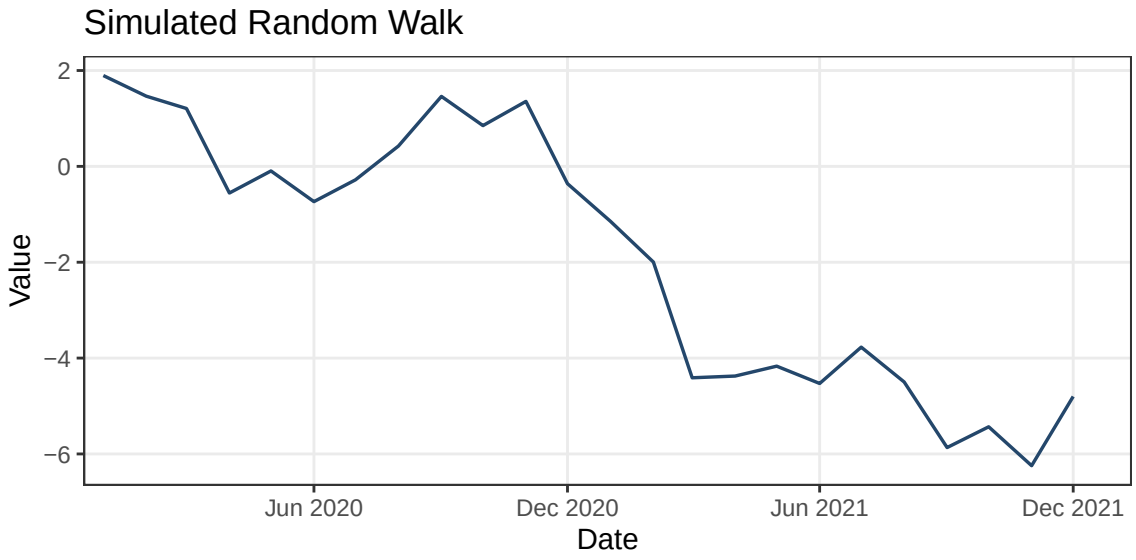
```
##      Jan   Feb   Mar   Apr   May   Jun   Jul   Aug   Sep   Oct   Nov   Dec
## 1   1.94 -0.80  0.51  0.90  0.57 -0.15  2.14 -0.13  2.85 -0.09  1.85  3.23
## 2 -1.96 -0.39 -0.19  0.90 -0.40 -3.76 -3.45  1.87 -0.43 -2.52 -0.24  1.72
```

Both objects contain the same values. The object `y` has frequency 1, while `z` has frequency 12, representing 12 observations per time unit. Interpreting the time unit as a year gives two years of monthly observations. Setting the frequency defines the time index; it does not introduce seasonality into the simulated data.

3 Visualizing a Simulated Random Walk

We construct a random walk by taking the cumulative sum of 24 independent standard normal increments. Monthly dates provide a calendar index for the plot.

```
mydata <- data.frame(
  time = seq.Date(
    from = as.Date("2020-01-01"),
    by = "month",
    length.out = 24
  ),
  value = cumsum(rnorm(24))
)
```



Although the increments have mean zero, their cumulative sum can display persistent upward or downward movements. Such movements do not imply that a deterministic trend was imposed.

4 Seasonal Decomposition Methods

An additive decomposition represents a time series as

$$Y_t = T_t + S_t + R_t,$$

where T_t is the trend, S_t is the seasonal component, and R_t is the remainder. A multiplicative decomposition instead uses

$$Y_t = T_t S_t R_t.$$

For positive observations, taking logarithms converts a multiplicative structure into an additive one.

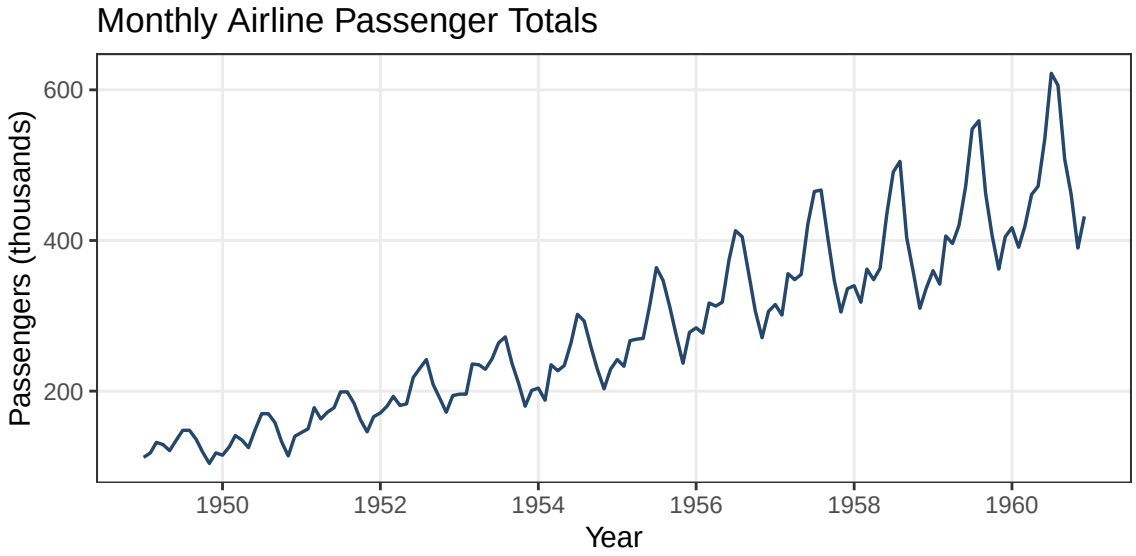
The function `decompose()` estimates the trend using moving averages and a seasonal pattern that repeats across cycles. The function `stl()` uses LOESS smoothing to separate trend, seasonality, and remainder.

STL can accommodate changing seasonality and optionally use robust fitting. In the application below, however, `s.window = "periodic"` imposes a fixed seasonal pattern on the log scale, and robust fitting is not enabled.

5 AirPassengers: Comparing Decomposition Methods

We analyze monthly international airline passenger totals from 1949 to 1960 using the built-in `AirPassengers` dataset.

```
ap_data <- data.frame(
  time = as.Date(as.yearmon(time(AirPassengers))),
  value = as.numeric(AirPassengers)
)
```



The series displays an upward trend and seasonal fluctuations whose amplitude increases with the level. This suggests a multiplicative structure. A logarithmic transformation reduces these differences in seasonal amplitude and makes an additive decomposition on the transformed scale a reasonable alternative.

We compare classical multiplicative decomposition, classical additive decomposition of log passenger totals, and STL decomposition of log passenger totals with periodic seasonality.

```
dec_mult <- decompose(
  AirPassengers,
```

```
  type = "multiplicative"
)

dec_add <- decompose(
  log(AirPassengers),
  type = "additive"
)

stl_fit <- stl(
  log(AirPassengers),
  s.window = "periodic"
)
```

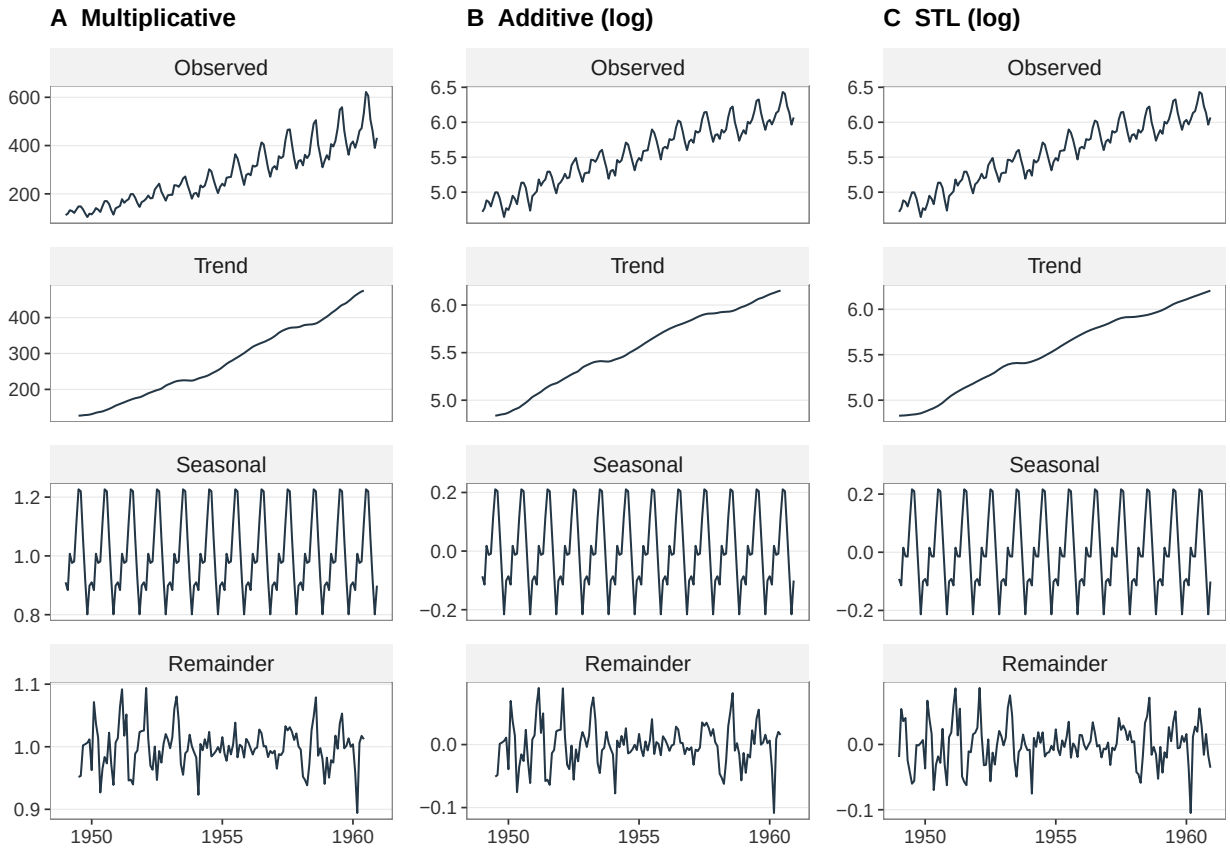


Figure 1. *AirPassengers decomposition. Columns show classical multiplicative decomposition (A), classical additive decomposition on the log scale (B), and periodic STL on the log scale (C). Vertical scales vary across panels; multiplicative components and log-scale components have different units.*

All three specifications impose a repeating seasonal pattern on their respective scales. The two log-scale decompositions differ in how they estimate the trend and seasonal components, rather than in whether seasonality is allowed to evolve.

Classical decomposition leaves missing trend and remainder values at the series endpoints because of its centered moving average. STL provides estimates across the full observed period.

The multiplicative remainder is expressed as a ratio around one, whereas the log-scale remainders are additive deviations around zero. Their magnitudes should therefore not be compared directly.

6 Atmospheric CO2: Trend and Seasonal Decomposition

We analyze monthly atmospheric carbon dioxide concentrations at Mauna Loa from 1959 to 1997, using the built-in `co2` dataset. Concentrations are expressed in parts per million (ppm).

We use classical additive decomposition to separate the long-term trend, a repeating annual seasonal pattern, and the remainder:

$$Y_t = T_t + S_t + R_t.$$

This specification assumes that seasonal effects have a constant absolute amplitude over time.

```
dec_co2 <- decompose(co2, type = "additive")

detrended <- co2 - dec_co2$trend
Yt_star <- detrended - dec_co2$seasonal
co2_remainder <- dec_co2$random
```

Subtracting the estimated trend leaves seasonal fluctuations and the remainder. Subtracting the seasonal component as well gives

$$Y_t^* = Y_t - \widehat{T}_t - \widehat{S}_t.$$

The first figure displays the observed series and its estimated components. Each panel has its own vertical scale.

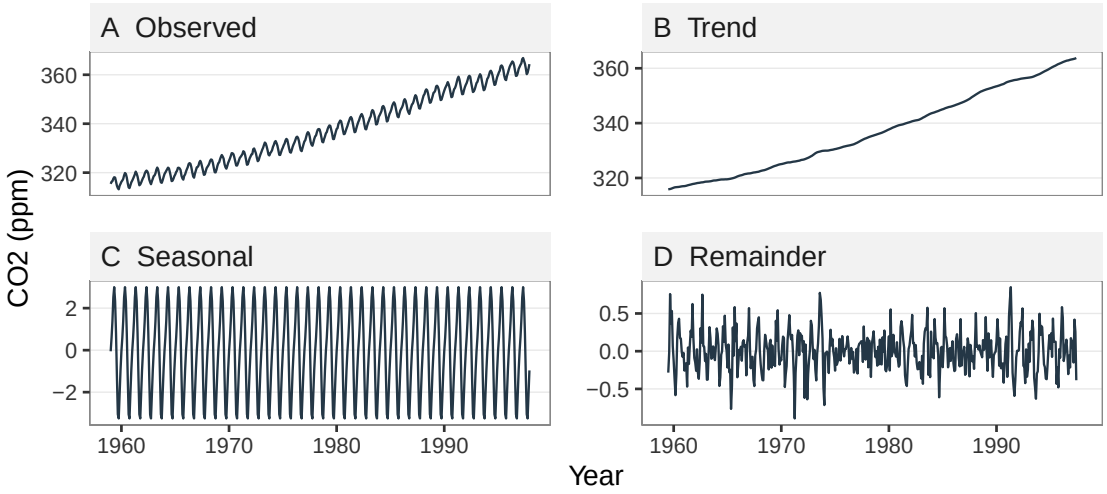


Figure 2. *Classical additive decomposition of monthly Mauna Loa CO2 concentrations. All panels are expressed in ppm, with separate vertical scales. The seasonal pattern repeats by construction; centered moving averages leave trend and remainder values unavailable at the endpoints.*

The estimated trend increases over the observation period. The seasonal component captures a recurring annual cycle. Its constant amplitude is imposed by classical decomposition, rather than independently established by the seasonal panel.

We next compare the original series with the results of removing the trend and seasonality. In the final panel, we overlay the decomposition remainder and the manually calculated Y_t^*.

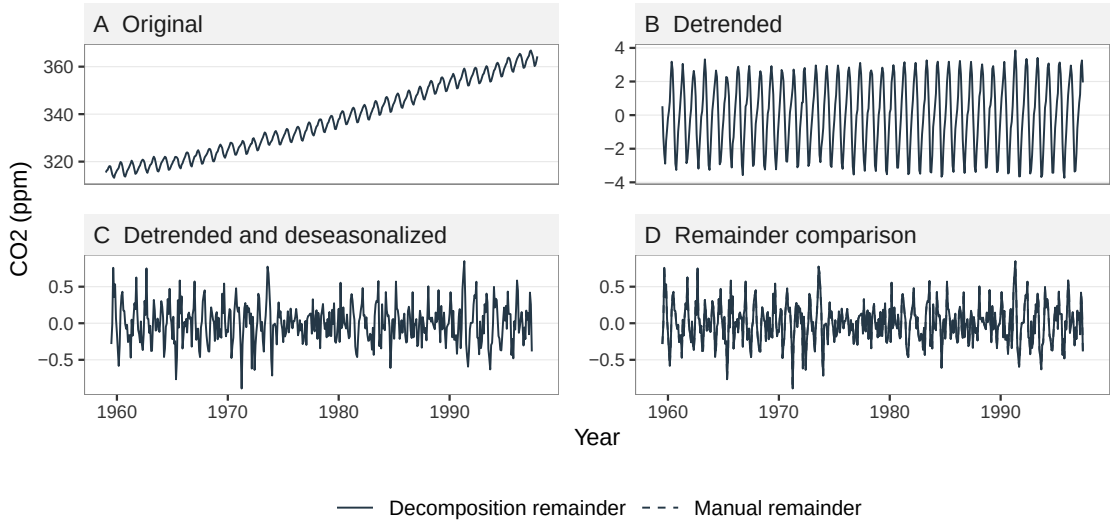


Figure 3. *Removing trend and seasonality from the CO2 series. Panel D overlays the decomposition remainder and the manually calculated remainder, which coincide by definition wherever the trend estimate is available. Vertical scales differ across panels.*

The detrended series retains the annual seasonal cycle. Removing the estimated seasonal component leaves fluctuations around zero.

The manually calculated series Y_t^* and the decomposition remainder coincide by construction. Their agreement checks the calculation, but does not establish that the remainder is white noise or that the decomposition fully captures the underlying dynamics.

7 UK Gas Consumption: Original and Logarithmic Scales

We analyze quarterly UK gas consumption from 1960 to 1986 using the built-in `UKgas` dataset. Seasonal fluctuations become larger as the level of the series increases, suggesting that a logarithmic transformation may be useful.

We compare classical additive decomposition of the original series with classical additive decomposition of its logarithm.

```
dec_add_uk <- decompose(UKgas, type = "additive")
dec_log_uk <- decompose(log(UKgas), type = "additive")
```

Both specifications impose a repeating seasonal pattern. On the original scale, seasonal effects are measured in absolute units. On the logarithmic scale, additive seasonal effects correspond to multiplicative factors on the original scale.

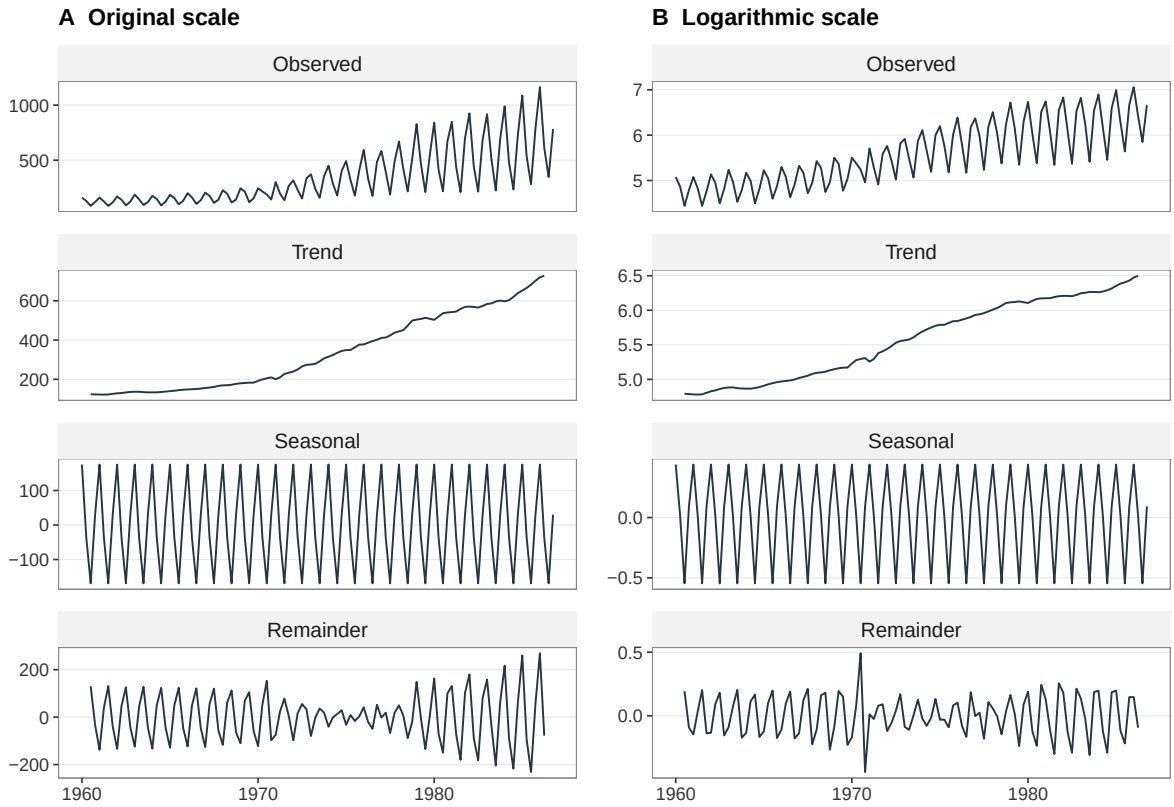


Figure 4. *Classical additive decomposition of UK gas consumption on the original scale (A) and logarithmic scale (B). Vertical scales vary across panels. Components on the two scales have different units.*

On the original scale, the remainder displays larger fluctuations toward the end of the sample. Taking logarithms reduces the contrast between early and late fluctuations, although visible structure remains in the remainder.

The repeating seasonal components are imposed by the decomposition method. Their appearance does not establish that seasonality is constant in the observed data. The logarithmic transformation addresses changes in scale, but does not allow the estimated seasonal pattern to evolve over time.

This comparison illustrates how the transformation affects the decomposition. It does not, by itself, establish that either specification produces an adequate remainder or better forecasts.

8 Nile River Flow: Simple Exponential Smoothing

We analyze annual Nile river flow at Aswan from 1871 to 1970 using the built-in `Nile` dataset. Flow is expressed in units of 10^8 cubic metres. We compare simple exponential smoothing with an estimated smoothing parameter and two fixed values: $\alpha = 0.1$ and $\alpha = 0.9$.

The level is updated according to

$$\ell_t = \alpha Y_t + (1 - \alpha)\ell_{t-1},$$

and the one-step-ahead fitted value is $\hat{Y}_{t|t-1} = \ell_{t-1}$. A larger α gives more weight to the latest observation. Trend and seasonal components are excluded.

```
hw_default <- HoltWinters(Nile, beta = FALSE, gamma = FALSE)
hw_01 <- HoltWinters(Nile, alpha = 0.1, beta = FALSE, gamma = FALSE)
hw_09 <- HoltWinters(Nile, alpha = 0.9, beta = FALSE, gamma = FALSE)
```

The first specification estimates α by minimizing the sum of squared one-step-ahead errors. The other two hold α fixed to illustrate how responsiveness changes.

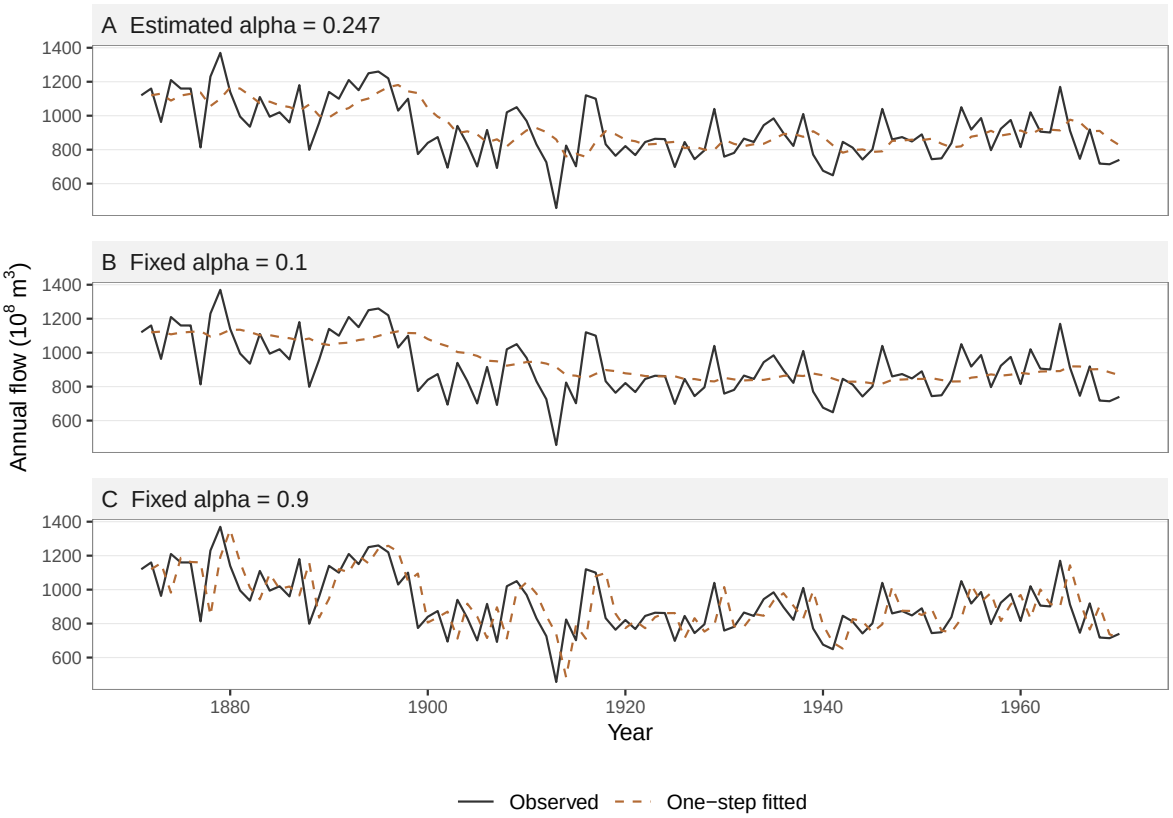


Figure 5. *Observed annual Nile river flow and one-step-ahead fitted values from simple exponential smoothing. All panels use the same axes. The smoothing parameter controls how quickly predictions respond to recent observations.*

With $\alpha = 0.1$, fitted values change gradually and respond slowly to shifts in the series. With $\alpha = 0.9$, they respond more rapidly to recent observations, but also reflect more of the year-to-year variation. The estimated parameter gives an intermediate response.

We summarize the one-step-ahead fitting errors using mean absolute percentage error:

$$\text{MAPE} = \frac{100}{n} \sum_t \left| \frac{Y_t - \hat{Y}_{t|t-1}}{Y_t} \right|,$$

Here, n = 99 is the number of observations with available fitted values, covering 1872–1970. The first observation, in 1871, initializes the level and has no corresponding one-step-ahead fitted value.

```r
nile_mape <- function(fit) {
  fitted_values <- fit$fitted[, "xhat"]

  observed_values <- window(
    Nile,
    start = start(fitted_values),
    end = end(fitted_values)
  )

  100 * mean(
    abs(
      (as.numeric(observed_values) - as.numeric(fitted_values)) /
        as.numeric(observed_values)
    )
  )
}

nile_results <- data.frame(
  Specification = c(
    "Estimated alpha",
    "Fixed alpha = 0.1",
    "Fixed alpha = 0.9"
  ),
  Alpha = c(hw_default$alpha, hw_01$alpha, hw_09$alpha),
  MAPE = vapply(nile_models, nile_mape, numeric(1))
)

knitr::kable(
  nile_results,
  digits = c(NA, 3, 2),
  col.names = c("Specification", "Alpha", "MAPE (%)"),
  row.names = FALSE,
  align = c("l", "r", "r")
)
```

Specification	Alpha	MAPE (%)
Estimated alpha	0.247	13.07
Fixed alpha = 0.1	0.100	13.48
Fixed alpha = 0.9	0.900	14.50

The estimated-alpha specification has the lowest MAPE among the three fits (13.07%), followed by alpha = 0.1 (13.48%) and alpha = 0.9 (14.50%). MAPE does not change monotonically across these three alpha values. The estimated alpha minimizes the sum of squared fitting errors, rather than MAPE.

These are in-sample fitting errors. No observations are held out, so the comparison does not establish which specification would forecast future river flow most accurately.

9 Italian COVID-19 Data: Daily Patterns and Smoothing

We examine cumulative reported cases, cumulative deaths, intensive care occupancy, and daily new reported cases in Italy. We then apply simple exponential smoothing to the daily case series.

The data file is stored in the repository. Dates are reconstructed from Italian day-month labels, starting in 2020 and advancing the year when the month sequence resets.

The observations cover 2020-02-20 to 2021-02-11. We display cumulative cases on both their original and natural logarithmic scales. Changes in log cumulative cases describe relative growth in the cumulative total, rather than growth in daily new cases.

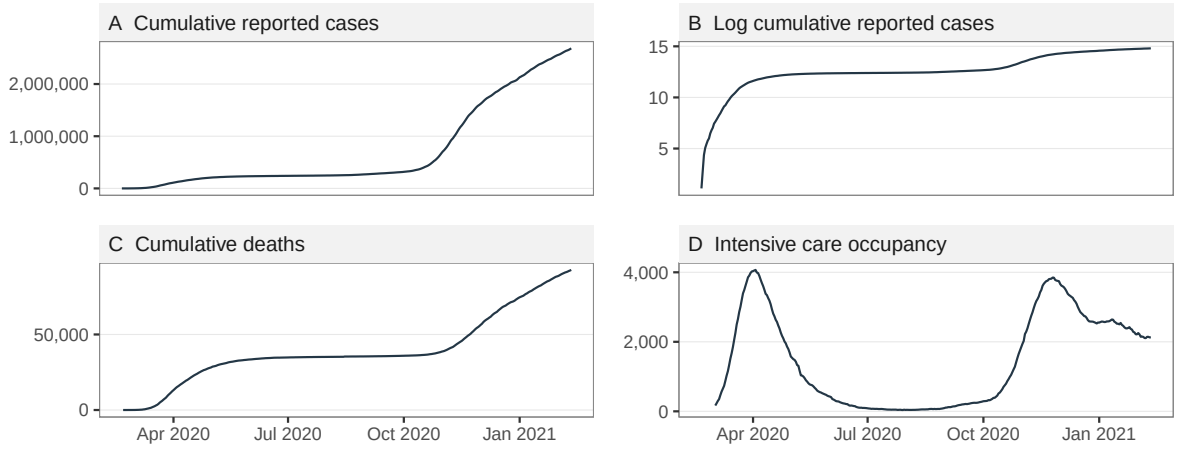


Figure 6. *Italian COVID-19 indicators. Panels show cumulative reported cases, their natural logarithm, cumulative deaths, and intensive care occupancy. Vertical scales and units differ across panels.*

Cumulative cases and deaths increase rapidly during two periods, separated by slower growth. Intensive care occupancy instead rises and falls, showing two distinct peaks.

These indicators describe different quantities: cumulative counts retain past events, while intensive care occupancy measures the number of patients occupying intensive care beds on each date.

Daily new reported cases are calculated as first differences of cumulative cases. Each difference is assigned to the later of the two dates.

```
nuovi_df <- data.frame(
  date = covid$date[-1],
  observed = nuovi_casi
)

nuovi_ts <- ts(nuovi_casi, frequency = 7)

hw <- HoltWinters(
  nuovi_ts,
  beta = FALSE,
  gamma = FALSE
)
```

We fit simple exponential smoothing without trend or seasonality. Setting `frequency = 7` defines seven observations per time unit; it does not introduce a weekly seasonal component when `gamma = FALSE`.

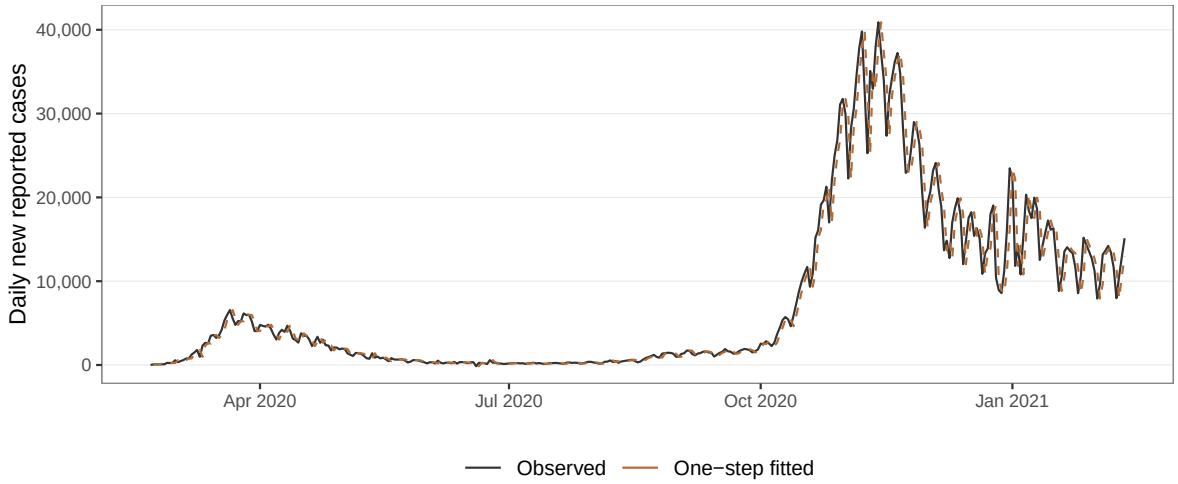


Figure 7. *Daily new reported cases and one-step-ahead fitted values from simple exponential smoothing. Daily cases are differences in cumulative reported cases and may reflect reporting practices or revisions as well as epidemic dynamics.*

The estimated smoothing parameter is $\alpha = 1.000$. Its proximity to one makes the level estimate highly responsive to the latest observation. Consequently, one-step-ahead fitted values closely follow the previous day's reported cases.

The daily series also displays recurring short-term fluctuations. The fitted model does not explicitly represent weekly seasonality or changes in epidemic growth.

This application illustrates smoothing and in-sample one-step prediction. No held-out forecast evaluation or prediction intervals are provided, so the plot does not establish forecasting accuracy or quantify uncertainty about future cases.